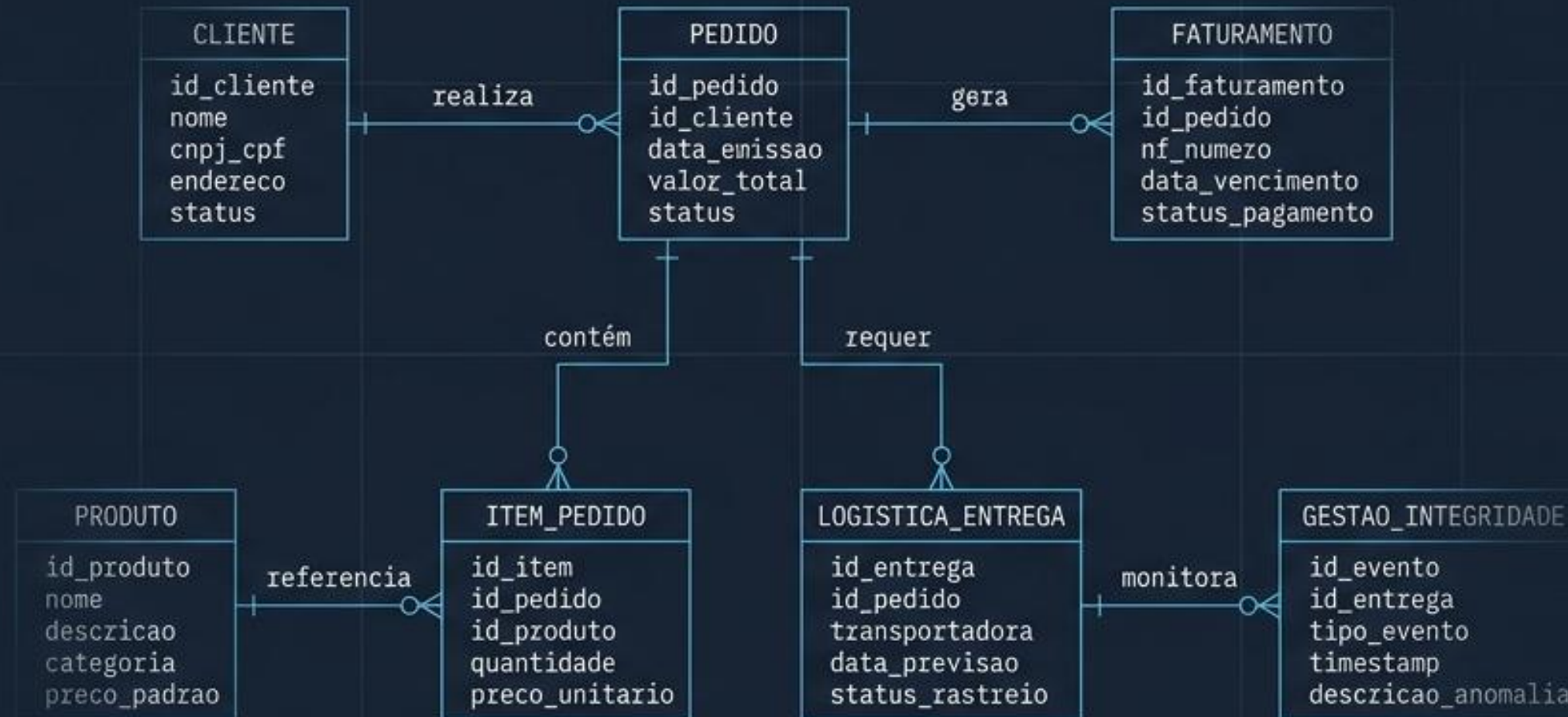
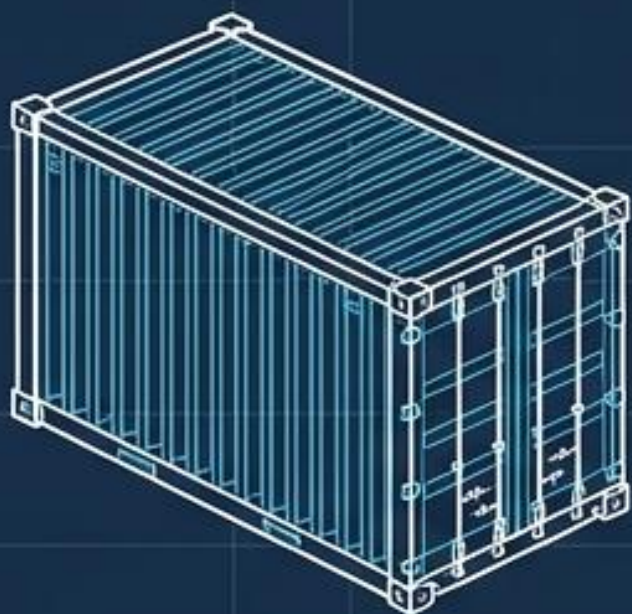


0 Núcleo Relacional

Arquiteturas de Dados e Big Data: Faturamento, Gestão e Integridade



A missão exige blindar operações financeiras contra falhas sistêmicas.



0 Desafio

Projetar um banco de dados transacional imune a falhas sistêmicas para uma operação logística de alta complexidade.



A Regra de Ouro

A consistência financeira não é negociável. Um erro no tráfego de dados significa perda de receita real.

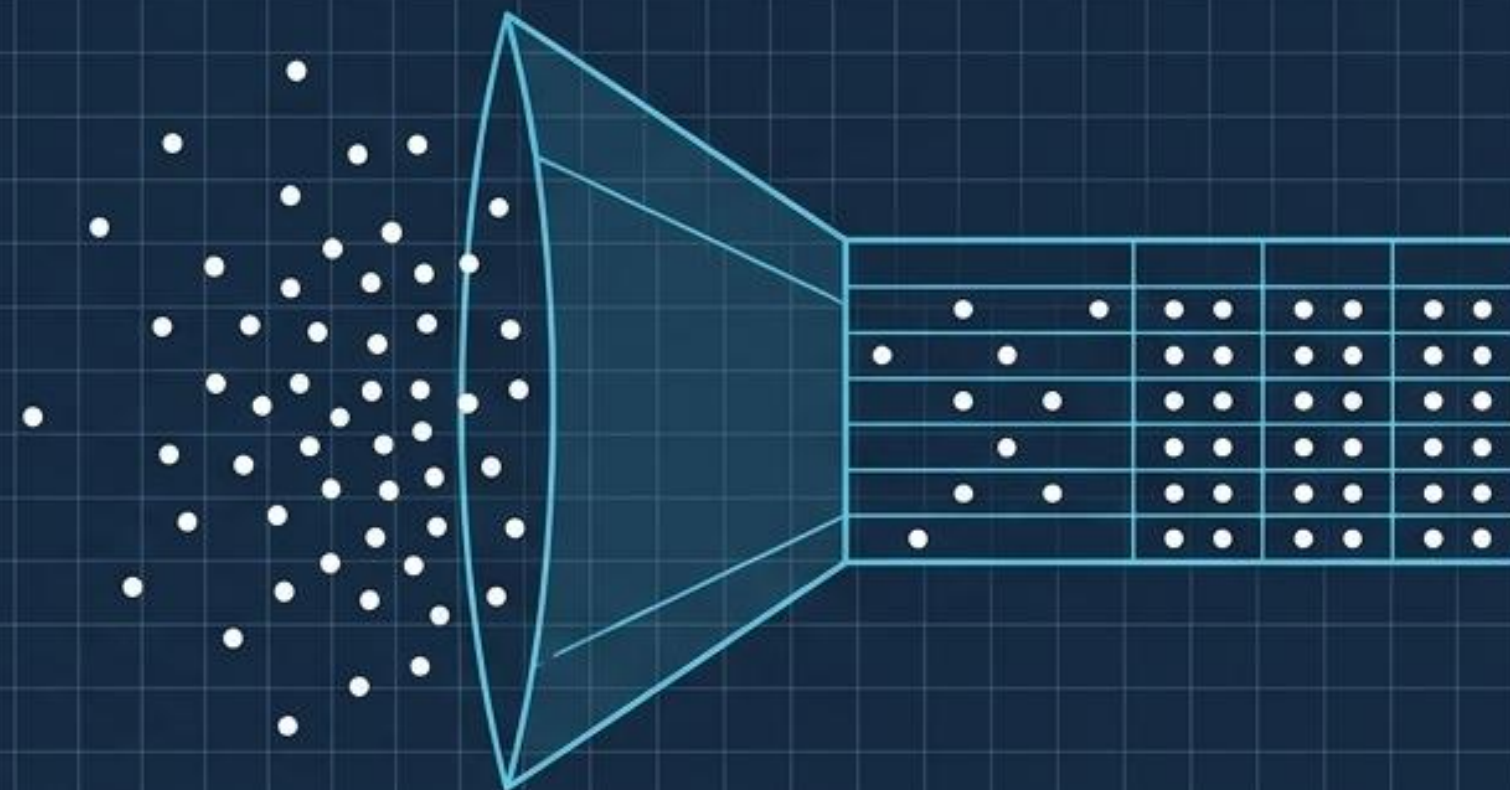


A Entrega Final

Um ambiente **PostgreSQL 16** provisionado via **Docker**, populado em escala e blindado por constraints estritas.

O paradigma relacional transforma dados caóticos em fatos matemáticos inegáveis.

Inventado por Edgar F. Codd, o modelo relacional substituiu a navegação manual de dados por uma estrutura baseada em teoria dos conjuntos matemáticos.



Edgar F. Codd (1970)

O que é uma Tupla?

Não é apenas uma linha em uma planilha. É a representação de um fato único, inegável e recuperável sobre o estado do negócio no tempo.

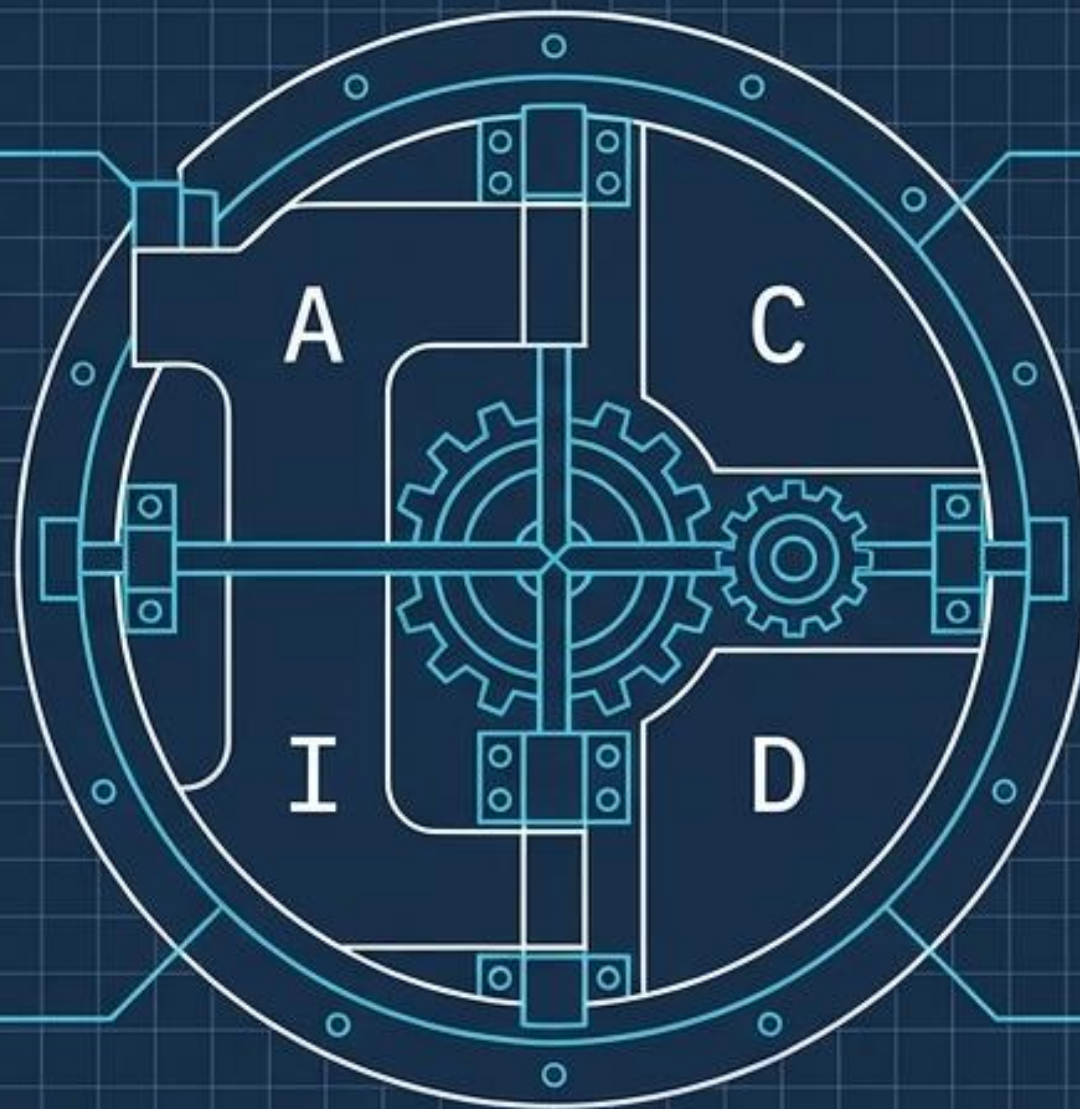
O Teorema ACID opera como um cofre de quatro travas simultâneas.



A (Atomicity): Tudo ou Nada. Operações indivisíveis.
Exemplo: Debitar da conta A e creditar na conta B ocorrem no mesmo microssegundo físico, ou nenhum ocorre.



I (Isolation): A Fila Invisível.
Múltiplas transações simultâneas não se sobrepõem nem enxergam dados inacabados umas das outras.



C (Consistency): De válido para válido. Nenhuma transação pode corromper as regras de negócio pré-definidas.



D (Durability): Esculpido em pedra. Sobrevivência garantida após queda de energia via WAL (Write Ahead Log).

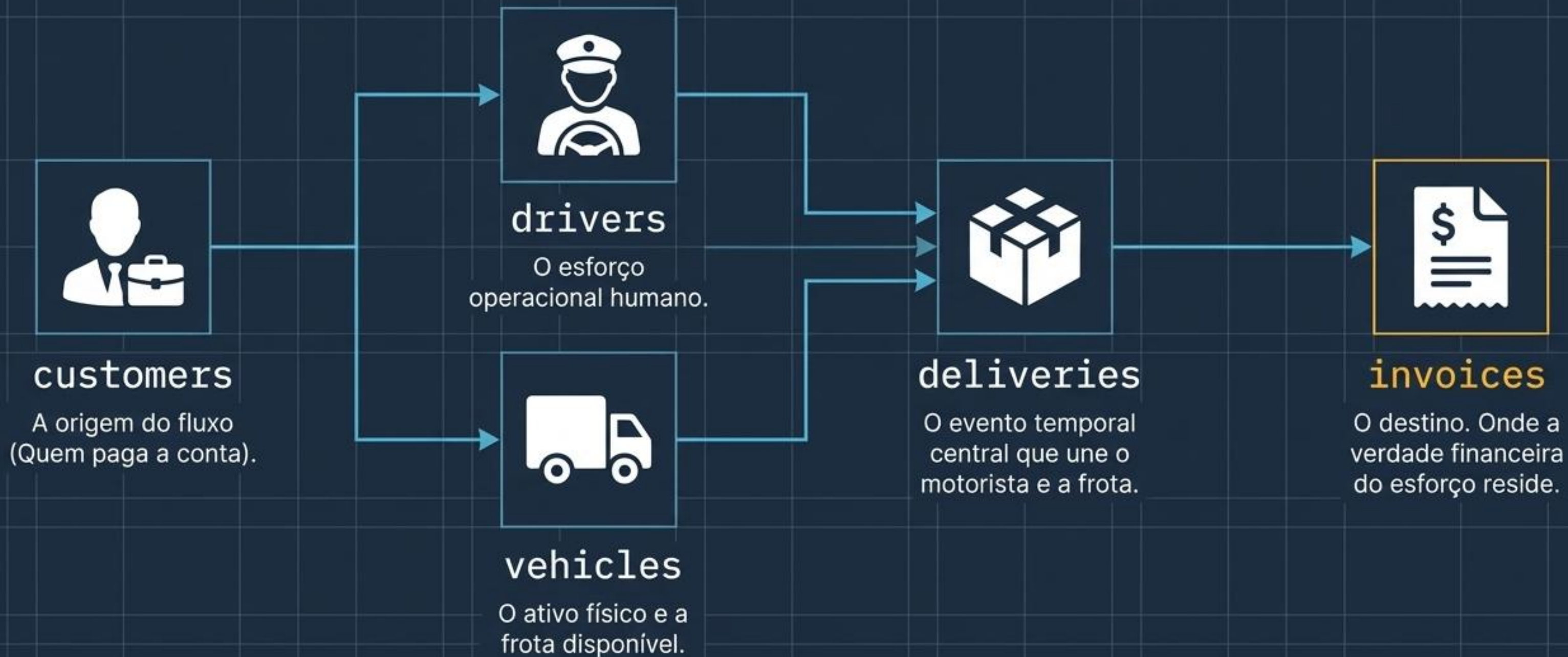
O risco financeiro dita a escolha entre agilidade NoSQL e rigor estrutural SQL.

Matriz de Risco Financeiro

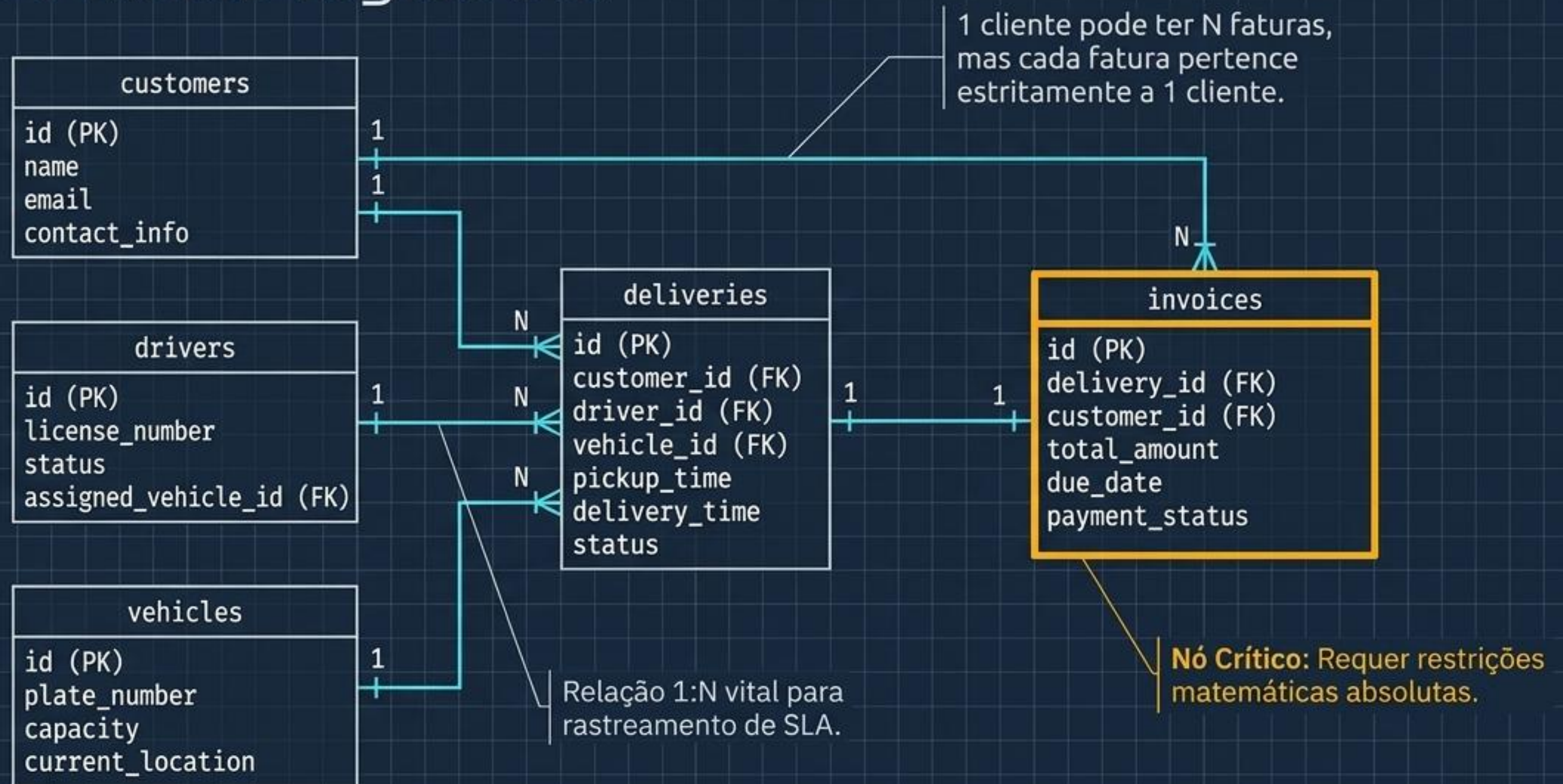
Vetor de Risco	NoSQL (Eventual Consistency)	Relacional SQL (ACID)
Perda de Dados (Data Loss)	Risco moderado em picos	Risco zero com ACID
Cobrança Duplicada (Double Billing)	Possível durante latência de sincronização	Bloqueado por Isolamento
Auditoria e SLA	Difícil rastreabilidade transacional	Histórico de mutação inegável

Insight: Para catálogos dinâmicos de produtos, NoSQL é ágil. Para faturamento, contratos e dinheiro humano, Relacional é sobrevivência.

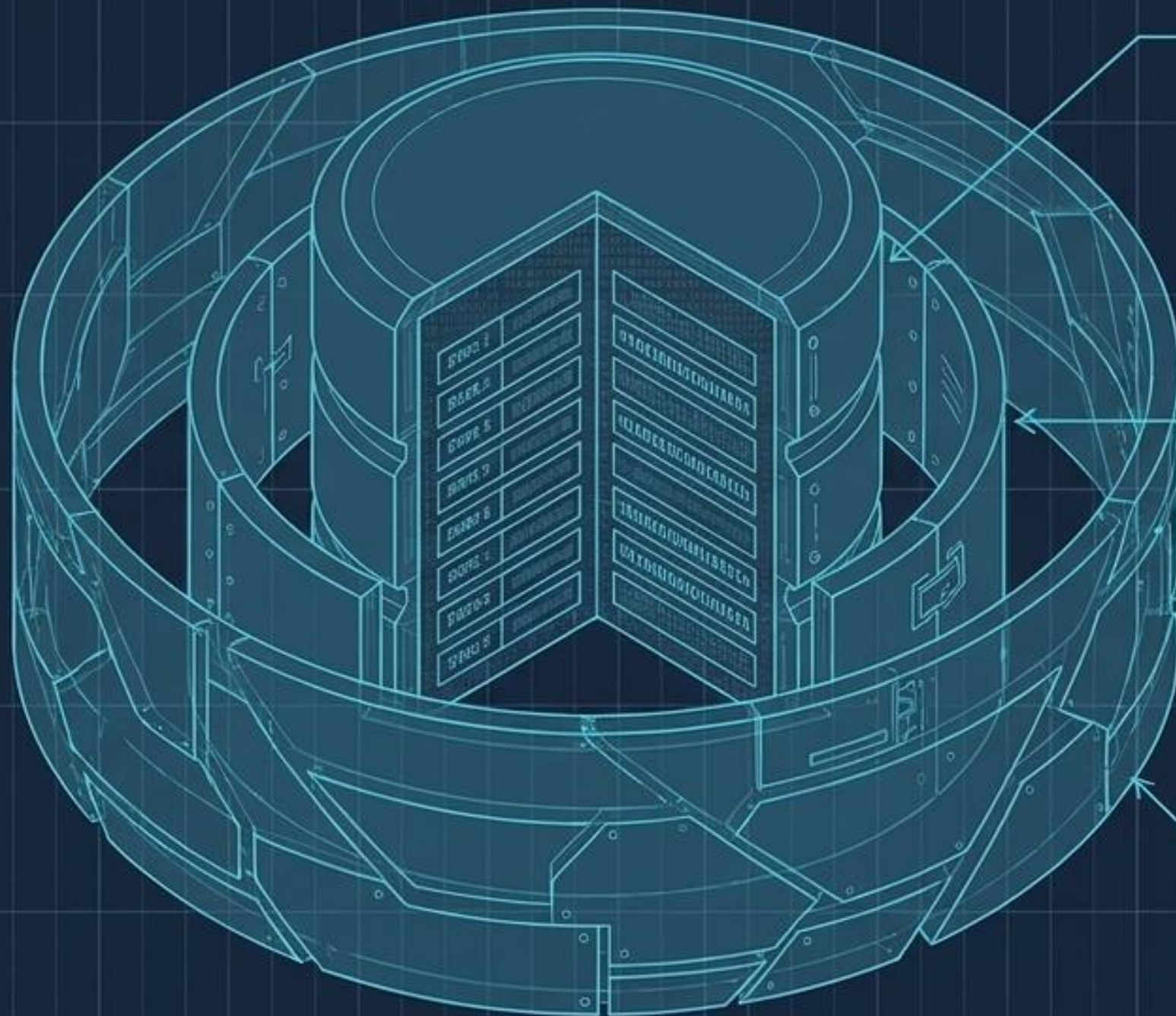
Mapeando a realidade física da SwiftTrack antes de escrever o código.



O diagrama de entidade-relacionamento define o motor logístico.



Três camadas de blindagem protegem as tabelas contra corrupção lógica.



Camada 1: Constraints (Regras Duras)

Impedem a entrada de estados impossíveis.

`NOT NULL, CHECK (amount > 0)`

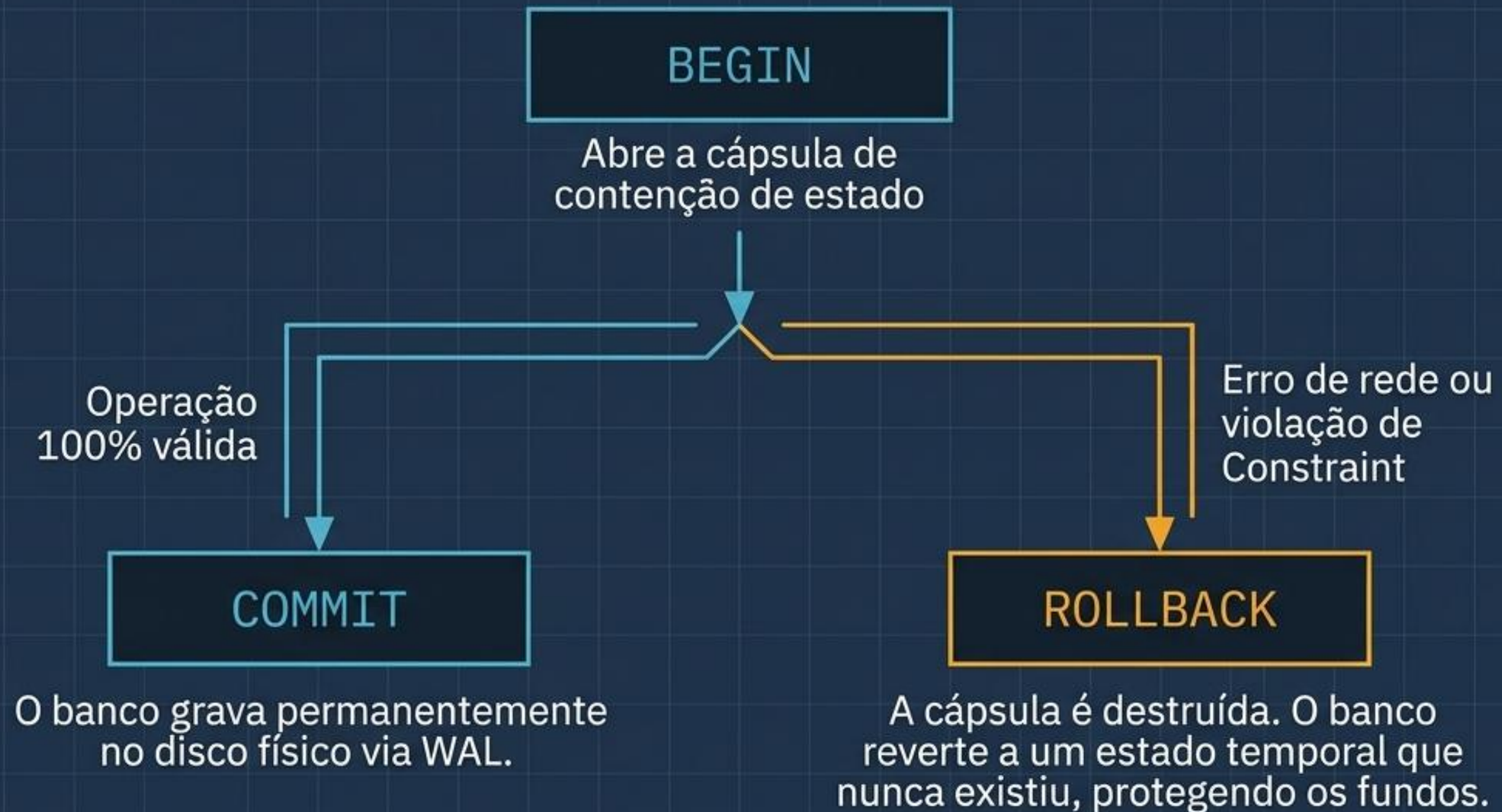
Camada 2: Foreign Keys (Integridade Referencial)

Garantem que dependências físicas existam. Nenhum motorista fantasma pode realizar uma entrega.

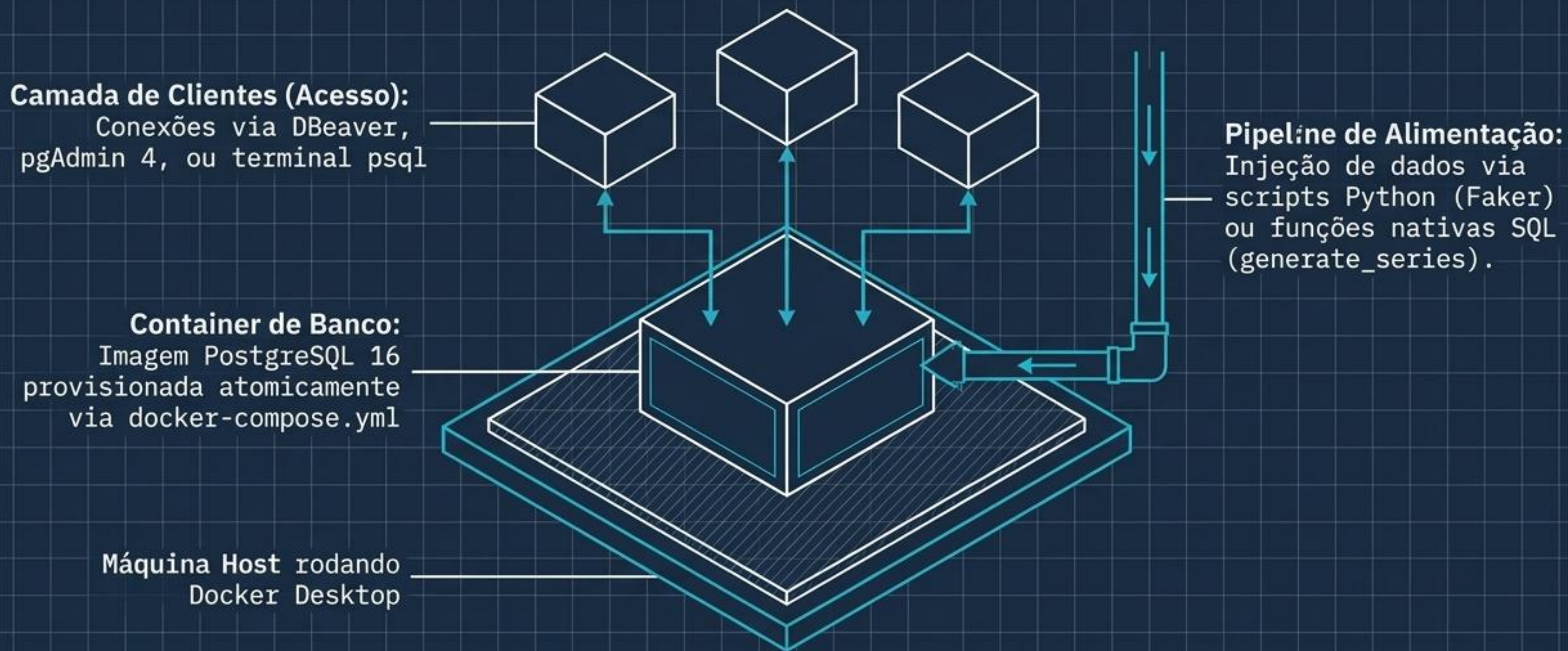
Camada 3: Índices B-Tree (Performance Segura)

Estrutura de busca logarítmica para escalar buscas de integridade sem sacrificar o tempo de resposta do sistema.

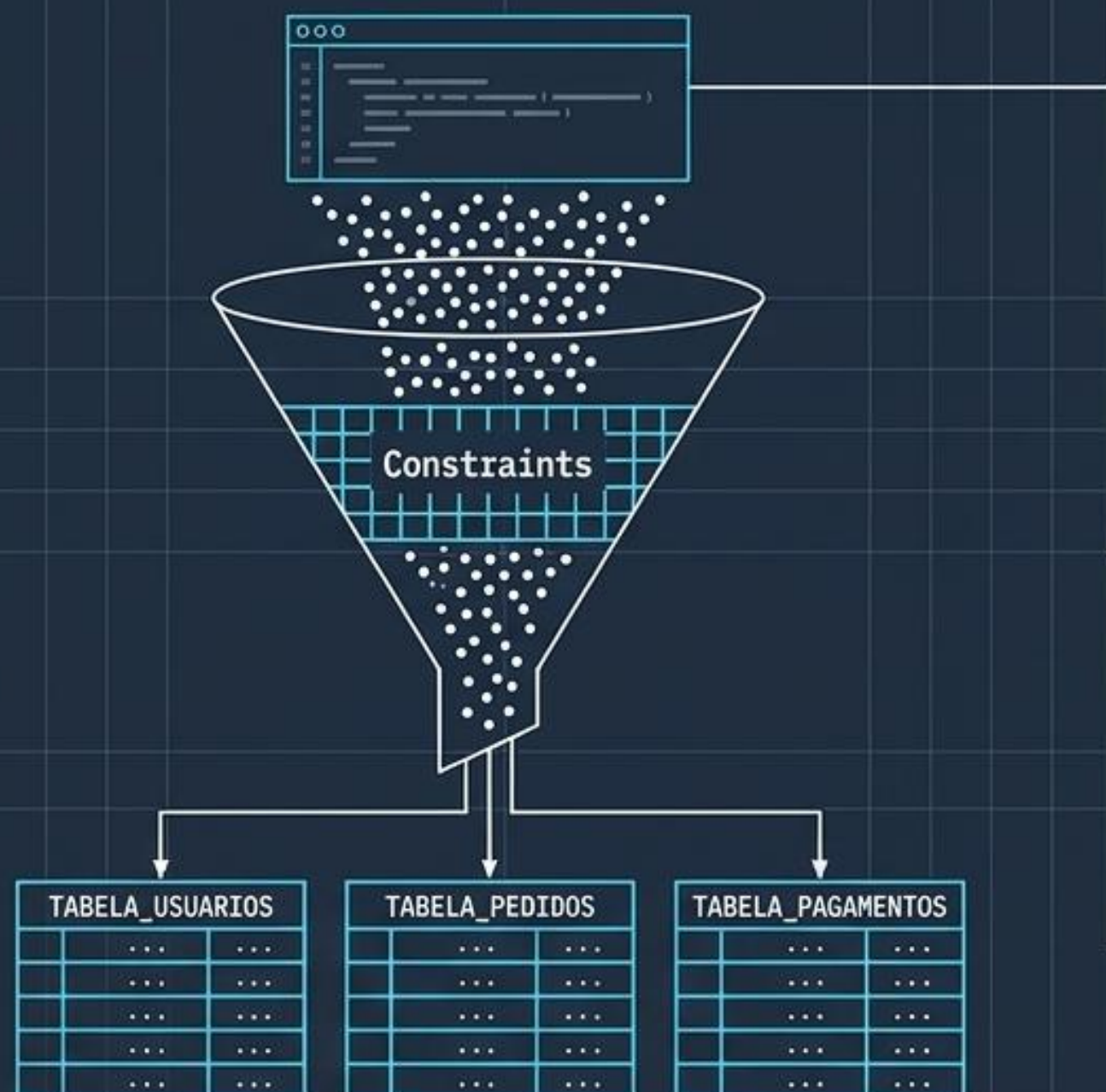
Transações explícitas garantem a reversão segura em caso de falhas críticas.



A arquitetura do ambiente de prática espelha infraestruturas de produção.



A validação do esquema exige estresse estrutural sob carga volumétrica sintética.



A Necessidade do Volume

Testar com 10 linhas esconde falhas de modelagem. O laboratório injetará 1.000 registros sintéticos (**Seeding**).

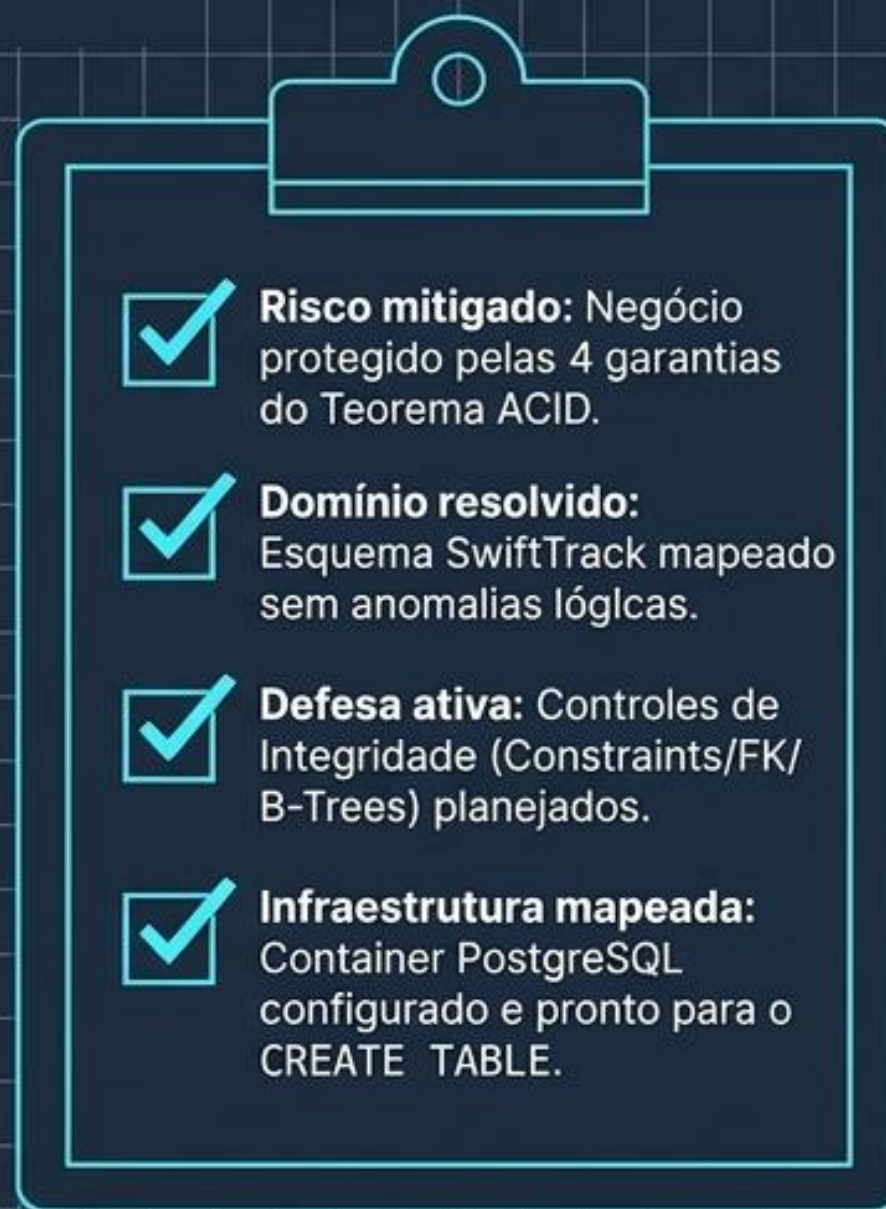
As Ferramentas de Injeção

Utilização de bibliotecas externas especializadas em dados falsos realistas (**Python/Faker**) contrapostas ao poder bruto das funções geradoras nativas do banco (**generate_series**).

O Teste Final

Se os dados passarem pelas restrições em escala, o modelo está validado para produção.

O blueprint está validado.
A fundação arquitetônica está
pronta para execução.



>_ Teoria concluída. Abra seu terminal. O lab prático de 3 horas começa agora.